

# STAR RPi5 Buildroot Implementation Primer

Custom Linux OS for Raspberry Pi 5 Robot Platform

STAR Development Team

December 2025

## Contents

# 1 Project Overview

## 1.1 Purpose

The `star-rpi5-buildroot` project creates a custom, minimal Linux distribution for the Raspberry Pi 5. It includes ROS 2, SLAM algorithms, navigation stack, and all dependencies needed for the STAR robot.

## 1.2 Current Implementation Status

| Component        | Status             | Notes                      |
|------------------|--------------------|----------------------------|
| Buildroot Base   | Complete           | 2025.02 LTS, ARM64         |
| Kernel Config    | Complete           | Linux 6.6 for Pi5          |
| WiFi Dual-Mode   | Complete           | Station + Access Point     |
| mDNS Discovery   | Complete           | star-robot.local via Avahi |
| OpenCV + Python  | Complete           | ML stack ready             |
| Camera Support   | Complete           | libcamera, V4L2            |
| systemd Services | Complete           | Network, gateway templates |
| ROS 2 Jazzy      | <b>Placeholder</b> | Directory structure only   |
| SLAM Toolbox     | <b>TODO</b>        | Package not integrated     |
| Nav2 Stack       | <b>TODO</b>        | Navigation packages needed |
| star-gateway     | <b>TODO</b>        | Spring Boot application    |
| LiDAR Drivers    | <b>TODO</b>        | rplidar_ros2, sick_scan_xd |

Table 1: Implementation Status Summary

### 1.3 Network Architecture

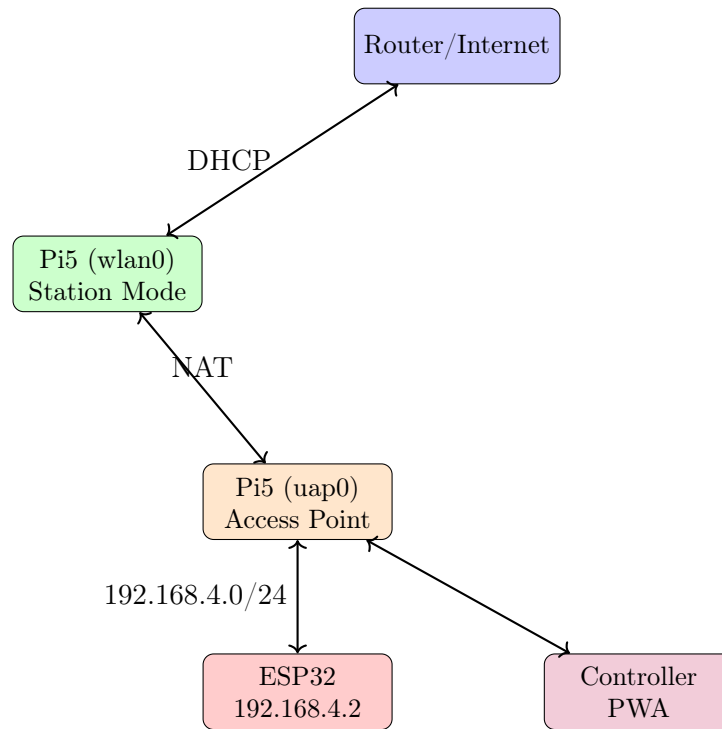


Figure 1: Dual WiFi Network Topology

## 2 Directory Structure

```

1 star-rpi5-buildroot/
2 |-- Makefile                # Build orchestration
3 |-- README.md               # User documentation
4 |-- Config.in               # External tree config
5 |-- external.desc           # Buildroot external desc
6 |-- external.mk             # Package includes
7 |
8 |-- configs/
9 |   |-- rpi5_64_defconfig    # Main Buildroot config
10 |   |-- config.txt           # Pi5 boot config
11 |   +-- busybox.config       # Minimal busybox
12 |
13 |-- package/
14 |   +-- ros2-jazzy/          # ROS 2 package (placeholder)
15 |       |-- Config.in
16 |       +-- ros2-jazzy.mk
17 |
18 |-- overlays/               # Rootfs customizations
19 |   |-- etc/
20 |       |-- hostapd/         # WiFi AP config
21 |       |-- wpa_supplicant/  # WiFi station config
22 |       |-- dnsmasq.d/       # DHCP for AP

```

```

23 | | | -- iptables/           # NAT rules
24 | | | -- systemd/system/   # Service files
25 | | +-- profile.d/         # Environment scripts
26 | +-- root/
27 |     +-- setup-network.sh # Interactive setup
28 |
29 +-- scripts/
30     |-- post-build.sh      # Enable services
31     |-- post-image.sh      # Generate SD image
32     +-- qemu-test.sh       # QEMU testing

```

## 3 Implementation Tasks

### 3.1 Phase 1: ROS 2 Integration

#### 3.1.1 Task 1.1: ROS 2 Jazzy Binary Integration

The current `ros2-jazzy` package is a placeholder. Implement actual binary installation:

##### Option A: Pre-built ARM64 Binaries

1. Download ROS 2 Jazzy ARM64 binaries from OSRF
2. Create Buildroot package that extracts to `/opt/ros/jazzy`
3. Handle dependencies (Python packages, shared libraries)
4. Test with `ros2 run demo_nodes_cpp talker`

Listing 1: `ros2-jazzy.mk` (Binary Installation)

```

1 ROS2_JAZZY_VERSION = jazzy
2 ROS2_JAZZY_SOURCE = ros2-jazzy-$(ROS2_JAZZY_VERSION)-linux-arm64.tar.bz2
3 ROS2_JAZZY_SITE = https://github.com/ros2/ros2/releases/download/...
4
5 define ROS2_JAZZY_INSTALL_TARGET_CMDS
6     mkdir -p $(TARGET_DIR)/opt/ros/jazzy
7     tar -xf $(@D)/$(ROS2_JAZZY_SOURCE) -C $(TARGET_DIR)/opt/ros/jazzy
8 endef
9
10 $(eval $(generic-package))

```

##### Option B: Build from Source (More Complex)

1. Add colcon build system to Buildroot
2. Cross-compile ROS 2 base packages
3. Handle CMake cross-compilation settings
4. Much longer build time (2+ hours)

### 3.1.2 Task 1.2: SLAM Toolbox Package

Create `package/slam-toolbox/`:

Listing 2: `slam-toolbox.mk`

```
1 SLAM_TOOLBOX_VERSION = jazzy
2 SLAM_TOOLBOX_SITE = $(call github,SteveMacenski,slam_toolbox,$(
   SLAM_TOOLBOX_VERSION))
3 SLAM_TOOLBOX_DEPENDENCIES = ros2-jazzy eigen boost
4
5 define SLAM_TOOLBOX_BUILD_CMDS
6     source /opt/ros/jazzy/setup.bash && \
7     cd $(@D) && colcon build --packages-select slam_toolbox
8 endef
9
10 $(eval $(generic-package))
```

### 3.1.3 Task 1.3: Nav2 Navigation Stack

Create `package/nav2/`:

Required packages:

- `nav2_bringup`
- `nav2_bt_navigator`
- `nav2_planner`
- `nav2_controller`
- `nav2_costmap_2d`
- `nav2_amcl`
- `dwb_core` (DWA local planner)

## 3.2 Phase 2: LiDAR Integration

### 3.2.1 Task 2.1: RPLiDAR Driver

Create `package/rplidar-ros2/`:

1. Clone `rplidar_ros2` from Slamtec
2. Build as ROS 2 package
3. Configure for UART: `/dev/ttyUSB0` at 460800 baud
4. Test with `ros2 launch rplidar_ros2 rplidar_c1_launch.py`

### 3.2.2 Task 2.2: SICK TiM561 Driver

Create `package/sick-scan-xd/`:

1. Clone `sick_scan_xd` from SICK AG
2. Build as ROS 2 package
3. Configure for Ethernet: `192.168.0.1:2112`
4. Test with `ros2 launch sick_scan_xd sick_tim_5xx.launch.py`

## 3.3 Phase 3: star-gateway Integration

### 3.3.1 Task 3.1: Java Runtime

Add OpenJDK 17 to Buildroot:

Listing 3: Add to `rpi5_64_defconfig`

```
1 BR2_PACKAGE_OPENJDK=y
2 BR2_PACKAGE_OPENJDK_VERSION_17=y
```

Note: OpenJDK build may fail on WSL. Use native Linux or pre-built binaries.

### 3.3.2 Task 3.2: Gateway Deployment

Update `overlays/etc/systemd/system/robot-gateway.service`:

Listing 4: `robot-gateway.service`

```
1 [Unit]
2 Description=STAR Robot Gateway
3 After=network-online.target ros2.service
4 Wants=network-online.target
5
6 [Service]
7 Type=simple
8 User=root
9 WorkingDirectory=/opt/star-gateway
10 ExecStart=/usr/bin/java -Xmx512m -jar star-gateway.jar
11 Environment="JAVA_HOME=/usr/lib/jvm/java-17-openjdk"
12 Environment="ROS_DISTRO=jazzy"
13 Environment="ROS_DOMAIN_ID=0"
14 Restart=always
15 RestartSec=10
16
17 [Install]
18 WantedBy=multi-user.target
```

### 3.3.3 Task 3.3: Gateway Build Script

Create deployment pipeline:

Listing 5: deploy-gateway.sh

```

1  #!/bin/bash
2  # Build and deploy star-gateway to Pi5
3
4  # Build JAR
5  cd ../star-gateway
6  ./gradlew bootJar
7
8  # Copy to Pi5
9  scp build/libs/star-gateway-*.jar root@star-robot.local:/opt/star-gateway/
   star-gateway.jar
10
11 # Restart service
12 ssh root@star-robot.local "systemctl restart robot-gateway"

```

### 3.4 Phase 4: System Configuration

#### 3.4.1 Task 4.1: ROS 2 Launch Files

Create overlays/opt/ros/star\_bringup/:

Listing 6: star\_robot.launch.py

```

1  from launch import LaunchDescription
2  from launch_ros.actions import Node
3
4  def generate_launch_description():
5      return LaunchDescription([
6          # LiDAR driver
7          Node(
8              package='rplidar_ros2',
9              executable='rplidar_node',
10             parameters=[{'serial_port': '/dev/ttyUSB0',
11                          'frame_id': 'laser_frame'}]
12         ),
13         # SLAM Toolbox
14         Node(
15             package='slam_toolbox',
16             executable='async_slam_toolbox_node',
17             parameters=['slam_params.yaml']
18         ),
19         # Nav2 stack
20         IncludeLaunchDescription(
21             PythonLaunchDescriptionSource('nav2_bringup/launch/
               navigation_launch.py'),
22             launch_arguments={'params_file': 'nav2_params.yaml'}.items()
23         ),
24     ])

```

#### 3.4.2 Task 4.2: SLAM Parameters

Create overlays/opt/ros/config/slam\_params.yaml:

Listing 7: slam\_params.yaml

```

1 slam_toolbox:
2   ros__parameters:
3     solver_plugin: solver_plugins::CeresSolver
4     ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
5
6     odom_frame: odom
7     map_frame: map
8     base_frame: base_link
9     scan_topic: /scan
10
11     resolution: 0.05           # 5cm per cell
12     max_laser_range: 12.0      # RPLiDAR C1 max
13     minimum_travel_distance: 0.3
14     minimum_travel_heading: 0.3
15
16     do_loop_closing: true
17     loop_search_maximum_distance: 3.0
18
19     # Performance tuning for RPi5
20     transform_publish_period: 0.05 # 20 Hz
21     map_update_interval: 0.5       # 2 Hz

```

### 3.4.3 Task 4.3: Nav2 Parameters

Create overlays/opt/ros/config/nav2\_params.yaml:

Listing 8: nav2\_params.yaml (DWA Controller)

```

1 controller_server:
2   ros__parameters:
3     controller_frequency: 20.0
4     FollowPath:
5       plugin: "dwb_core::DWBLocalPlanner"
6       max_vel_x: 0.5
7       min_vel_x: -0.3
8       max_vel_theta: 1.0
9       acc_lim_x: 1.0
10      acc_lim_theta: 2.0
11      decel_lim_x: -1.5
12      vx_samples: 20
13      vtheta_samples: 40
14      sim_time: 1.5
15      critics: ["RotateToGoal", "Oscillation", "BaseObstacle",
16               "GoalAlign", "PathAlign", "PathDist", "GoalDist"]

```

## 4 Build Instructions

### 4.1 Prerequisites

```

1 # Ubuntu/Debian dependencies
2 sudo apt-get install build-essential libncurses5-dev git \

```

```
3     bc bison flex libssl-dev make unzip wget cpio rsync
4
5 # Disk space: ~20GB free
6 # Build time: 1-2 hours (first build)
```

## 4.2 Build Commands

```
1 cd star-rpi5-buildroot
2
3 # Full build (download + configure + build)
4 make build
5
6 # Or step-by-step:
7 make download      # Get Buildroot 2025.02
8 make configure     # Apply our config
9 make               # Build everything
10
11 # Output: buildroot-2025.02/output/images/sdcard.img
```

## 4.3 Flash to SD Card

```
1 # Find SD card device (e.g., /dev/sdb)
2 lsblk
3
4 # Flash image (CAUTION: double-check device!)
5 sudo dd if=buildroot-2025.02/output/images/sdcard.img \
6     of=/dev/sdX bs=4M status=progress
7 sync
```

## 4.4 First Boot Setup

```
1 # Connect via Ethernet or serial console
2 ssh root@<IP_ADDRESS> # No password
3
4 # Or via mDNS (after network configured)
5 ssh root@star-robot.local
6
7 # Run interactive network setup
8 /root/setup-network.sh
9 # Choose option 8 for full setup
```

# 5 Testing

## 5.1 QEMU Testing

Test without hardware using QEMU:

```
1 # Serial console boot
2 make qemu
3
4 # Graphical mode (VNC)
5 make qemu-graphical
6
7 # SSH into QEMU instance
8 make qemu-ssh
```

Note: QEMU uses generic ARM64 machine, not Pi5-specific. Good for testing systemd, networking, userspace.

## 5.2 Hardware Testing Checklist

1. Boot Pi5, verify serial console output
2. Check WiFi station connects to router
3. Check WiFi AP creates STAR-Robot-AP network
4. Verify ESP32 can connect to AP
5. Test mDNS: `ping star-robot.local`
6. Test SSH access
7. Verify camera: `libcamera-hello`
8. Test ROS 2: `ros2 topic list`
9. Run SLAM with LiDAR
10. Test star-gateway API endpoints

## 6 Troubleshooting

### 6.1 Common Issues

**Build fails on WSL** OpenJDK cross-compilation doesn't work. Use native Linux or remove Java from config.

**WiFi AP not starting** Check `systemctl status hostapd`. May need to manually create `uap0` interface.

**ROS 2 not found** The `ros2-jazzy` package is a placeholder. Install ROS 2 manually after first boot.

**SSH permission denied** Default config allows empty root password. If changed, use the password you set.

## 6.2 Debugging Commands

```
1 # Check service status
2 systemctl status hostapd dnsmasq-ap wifi-nat wpa_supplicant@wlan0
3
4 # View logs
5 journalctl -u robot-gateway -f
6
7 # Network interfaces
8 ip addr show
9 iwconfig
10
11 # ROS 2 environment
12 source /opt/ros/jazzy/setup.bash
13 ros2 doctor
```

## 7 References

- Buildroot Manual: <https://buildroot.org/downloads/manual/manual.html>
- ROS 2 Jazzy Installation: <https://docs.ros.org/en/jazzy/Installation.html>
- SLAM Toolbox: [https://github.com/SteveMacenski/slam\\_toolbox](https://github.com/SteveMacenski/slam_toolbox)
- Nav2 Documentation: <https://docs.nav2.org/>
- RPLiDAR ROS 2: [https://github.com/Slamtec/rplidar\\_ros](https://github.com/Slamtec/rplidar_ros)
- STAR FTP Section 17: Software Implementation
- STAR Software Architecture Primer: Sections 5, 14